



HCM-UTE

TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

HCMC University of Technology and Engineering

[SOEN330679] CÔNG NGHỆ PHẦN MỀM

CHƯƠNG 6: CÀI ĐẶT VÀ KIỂM THỬ PHẦN MỀM (01)

ThS. Võ Quang Phúc

MỤC TIÊU

1. **MT1.** Giải thích được sự tương ứng giữa mô hình BCE (Boundary–Control–Entity) và ba tầng kiến trúc trong triển khai thực tế.
2. **MT2.** Cài đặt được ít nhất hai Use Case theo đúng kiến trúc 3-tier, tham chiếu trực tiếp từ Sequence Diagram, Class Diagram và ERD.
3. **MT3.** Nắm vững các nguyên tắc cài đặt phần mềm và biết cách chọn các công nghệ lập trình phù hợp với dự án.
4. **MT4.** Hiểu rõ kiến trúc phân tầng (3 tầng, MVC) và cách phân loại các công nghệ lập trình hiện đại theo vai trò kiến trúc.
5. **MT5.** Hiểu và vận dụng các tiêu chuẩn chất lượng mã nguồn.

NỘI DUNG

1. Giai đoạn cài đặt phần mềm

- a) Định nghĩa
- b) Ba nguyên tắc nền tảng trong cài đặt

2. Các công nghệ lập trình hiện đại

- a) Phân loại công nghệ theo vai trò kiến trúc
- b) Kiến trúc phân tầng, MVC và BCE

3. Ánh xạ từ thiết kế sang mã nguồn

a) Từ Class Diagram

b) Từ Sequence Diagram

4. Tiêu chuẩn chất lượng mã nguồn

- a) Bốn nguyên tắc lập trình cốt lõi
- b) Quản lý phiên bản với Git
- c) ISO/IEC 25010:2023

5. Thực hành

1. Giai đoạn cài đặt phần mềm

1.1. Định nghĩa

Cài đặt phần mềm (Software Construction / Implementation)

Theo IEEE SWEBOOK v3.0, Software Construction là giai đoạn tạo ra phần mềm hoạt động thông qua kết hợp giữa:

- (1) lập trình chi tiết (detailed coding),
- (2) kiểm thử đơn vị và tích hợp (unit & integration testing),
- (3) gỡ lỗi (debugging), và
- (4) xem xét mã nguồn (code review).

Đây là giai đoạn sản xuất trực tiếp ra sản phẩm, kết quả của nó là phần mềm có thể thực thi được, không phải tài liệu hay bản vẽ

1. Giai đoạn cài đặt phần mềm

1.1. Định nghĩa

Lưu ý: Theo nghiên cứu của Barry Boehm (1981) và được xác nhận lại bởi nhiều nghiên cứu hiện đại, chi phí sửa lỗi ở giai đoạn cài đặt cao gấp 10 lần so với sửa lỗi ở giai đoạn yêu cầu, và cao gấp 100 lần nếu lỗi bị phát hiện sau khi triển khai. Đây là lý do tại sao thiết kế cẩn thận ở các chương trước lại có ý nghĩa kinh tế trực tiếp.



Chi phí tương đối để sửa cùng một lỗi yêu cầu, theo thời điểm phát hiện (Boehm, 1981; số liệu điển hình cho hệ thống lớn).

1. Giai đoạn cài đặt phần mềm

1.1. Định nghĩa

- ❑ Trong mô hình phát triển phần mềm hiện đại, giai đoạn cài đặt không phải là bước đơn thuần “gõ code” mà là một công đoạn kỹ thuật đòi hỏi kỹ năng tổng hợp: nắm vững thiết kế hệ thống, hiểu nguyên tắc kiến trúc, làm chủ công nghệ được chọn, và tuân thủ chuẩn chất lượng mã nguồn của nhóm.
- ❑ Theo Standish Group CHAOS Report 2020, chỉ khoảng 31% dự án phần mềm thành công (đúng hạn, đúng ngân sách, đủ tính năng); 50% gặp vấn đề (challenged) và 19% thất bại. Cài đặt lệch khỏi thiết kế là một nguồn tích lũy nợ kỹ thuật (technical debt) khiến dự án ngày càng khó kiểm soát.

1. Giai đoạn cài đặt phần mềm

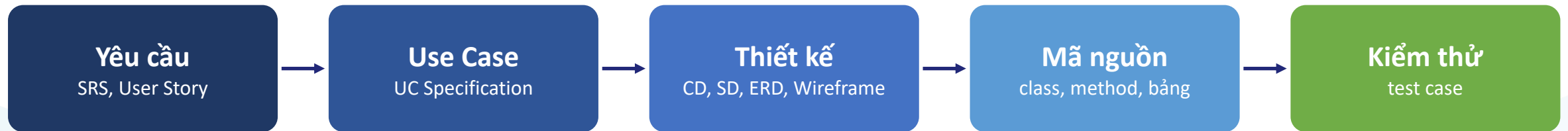
1.2. Ba nguyên tắc nền tảng trong Cài đặt

- ❑ Nguyên tắc 1: **TRACEABILITY** (Khả năng Truy nguyên)
- ❑ Nguyên tắc 2: **FIDELITY** (Trung thành với Thiết kế)
- ❑ Nguyên tắc 3: **INCREMENTALITY** (Cài đặt Tăng tiến)

1. Giai đoạn cài đặt phần mềm

1.2.1. Nguyên tắc 1: TRACEABILITY (Khả năng Truy nguyên)

- ❑ Mỗi thành phần trong mã nguồn (từng class, từng method, từng module) phải có thể truy nguyên ngược về ít nhất một element trong hệ thống sản phẩm/tài liệu thiết kế.
- ❑ Nguyên tắc này được quy định bởi IEEE 29148:2018 như một yêu cầu bắt buộc trong kỹ thuật phần mềm chuyên nghiệp.



← Truy nguyên ngược: từ một dòng code tìm lại được thiết kế và yêu cầu đã sinh ra nó

1. Giai đoạn cài đặt phần mềm

1.2.1. Nguyên tắc 1: TRACEABILITY (Khả năng Truy nguyên)

Thành phần code	Truy nguyên về tài liệu nào
Class `Order` có các thuộc tính: orderId, tableNumber, items[], totalAmount, status	Class Diagram: class `Order` với các attribute và operation
Method `calculateTotal()`: Method tính tổng tiền, được gọi đúng thứ tự theo SD sau khi tất cả items được thêm	Operation `calculateTotal(): Decimal` trong Class Diagram + message trong Sequence Diagram
Bảng `orders` trong CSDL: có cột orderId, table_id (FK), created_at, total_amount, status	Entity ORDERS trong ERD với đầy đủ các cột và khóa ngoại
Component ` <orderform>`: Form có input số bàn, danh sách món, nút Thêm và Xác nhận</orderform>	Wireframe màn hình “Tạo đơn hàng” trong Interface Design

1. Giai đoạn cài đặt phần mềm

1.2.2. Nguyên tắc 2: FIDELITY (Trung thành với Thiết kế)

- ❑ Mã nguồn phải phản ánh trung thành kiến trúc đã được quyết định trong giai đoạn thiết kế.
- ❑ Nếu trong quá trình cài đặt phát sinh nhu cầu thay đổi thiết kế (điều hoàn toàn bình thường và thực tế), quy trình đúng là:
 - (1) Dừng lại,
 - (2) cập nhật tài liệu liên quan,
 - (3) sau đó mới viết code theo thiết kế mới.
 - (4) Không được phép “sửa code rồi quên cập nhật sơ đồ” → đây là nguồn gốc của hầu hết các vấn đề bảo trì dài hạn trong các dự án thực tế.

1. Giai đoạn cài đặt phần mềm

1.2.3. Nguyên tắc 3: INCREMENTALITY (Cài đặt tăng tiến)

- ❑ Nên cài đặt và kiểm thử từng Use Case một cách hoàn chỉnh (Model + Controller + View + Database) trước khi chuyển sang UC tiếp theo.
- ❑ Cách tiếp cận này trái với cách làm phổ biến nhưng sai lầm là: “làm hết tất cả Model → làm hết tất cả Controller → làm hết tất cả View”.
- ❑ Kinh nghiệm thực tế cho thấy cách tiếp cận tầng-trước-tầng dẫn đến: phát hiện lỗi muộn, khó debug vì không biết lỗi ở tầng nào, và không có milestone rõ ràng để đánh giá tiến độ.

1. Giai đoạn cài đặt phần mềm

1.2.3. Minh họa: cài đặt tầng-trước-tầng và cài đặt theo lát dọc (vertical slice)

✗ Tầng trước tầng (không nên)

	UC-01	UC-03	UC-07
View (UI)	10	11	12
Controller / Service	7	8	9
Entity / Repository	4	5	6
Database	1	2	3

Làm hết Database, rồi hết Entity, Controller, View: chỉ chạy thử được toàn bộ sau bước 12, lỗi tích hợp lộ ra muộn.

✓ Hoàn thiện từng Use Case (nên)

	UC-01	UC-03	UC-07
View (UI)	4	8	12
Controller / Service	3	7	11
Entity / Repository	2	6	10
Database	1	5	9
	Demo được	Demo được	Demo được

Sau mỗi 4 bước có một Use Case chạy được end-to-end để kiểm thử và làm mốc tiến độ.

2. Các công nghệ lập trình hiện đại

2.1. Phân loại công nghệ theo vai trò kiến trúc

Lớp kiến trúc	Vai trò chức năng	Công nghệ phổ biến
Presentation Layer (Frontend / UI)	Hiển thị giao diện người dùng; nhận, validate input phía client; phản hồi kết quả theo thời gian thực	React 18, Vue 3, Next.js 14, Nuxt 3, Flutter 3.x, SwiftUI, Jetpack Compose
Business Logic Layer (Backend / Application Server)	Xử lý toàn bộ quy tắc nghiệp vụ; validate phía server; điều phối giao dịch và luồng xử lý	Node.js/Express 4, NestJS, Spring Boot 3 (Java), Django 5 (Python), Laravel 11 (PHP), .NET 8 (C#), Go/Gin
Data Access Layer (ORM / Repository)	Trừu tượng hoá các thao tác CSDL; ánh xạ đối tượng — bảng; quản lý transaction và connection pool	Prisma ORM (JS/TS), Hibernate (Java), SQLAlchemy (Python), Eloquent (PHP), TypeORM, Entity Framework (.NET)

2. Các công nghệ lập trình hiện đại

2.1. Phân loại công nghệ theo vai trò kiến trúc

Lớp kiến trúc	Vai trò chức năng	Công nghệ phổ biến
Database Layer (Lưu trữ)	Lưu trữ dữ liệu bền vững; đảm bảo tính toàn vẹn (ACID); hỗ trợ truy vấn phức tạp và indexing	PostgreSQL 16, MySQL 8, MongoDB 7 (document), Redis 7 (in-memory/cache), SQLite (phát triển nội bộ)
API Layer (Giao tiếp)	Định nghĩa hợp đồng giao tiếp giữa Frontend và Backend; xử lý phân quyền, rate limiting, versioning	RESTful API (phổ biến nhất), GraphQL (flexible query), gRPC (hiệu suất cao), WebSocket (real-time)
DevOps / Triển khai	Đóng gói ứng dụng; CI/CD tự động hoá; môi trường phát triển nhất quán; giám sát và logging	Docker, Docker Compose, GitHub Actions, GitLab CI, Vercel, Railway, AWS EC2/ECS, Render

2. Các công nghệ lập trình hiện đại

2.2. Kiến trúc phân tầng (Layered Architecture)

- ❑ Kiến trúc 3-tầng (3-Tier / Layered Architecture): là mô hình tổ chức phần mềm tách biệt hệ thống thành ba tầng độc lập về logic, mỗi tầng có trách nhiệm rõ ràng và chỉ giao tiếp với tầng liền kề.
- ❑ Ba tầng bao gồm:
 - (1) **Presentation Tier**: xử lý giao diện và tương tác người dùng;
 - (2) **Business Logic Tier**: chứa toàn bộ quy tắc nghiệp vụ, validation và điều phối;
 - (3) **Data Tier**: quản lý lưu trữ và truy xuất dữ liệu. Mô hình này đảm bảo tính độc lập (independence), khả năng kiểm thử từng phần (testability), và dễ bảo trì (maintainability).

2. Các công nghệ lập trình hiện đại

2.2. Sơ đồ kiến trúc 3 tầng



Chỉ giao tiếp với tầng liền kề

Presentation không truy cập CSDL trực tiếp (đường đỏ). Mọi thao tác dữ liệu đi qua tầng nghiệp vụ.

Layer và Tier

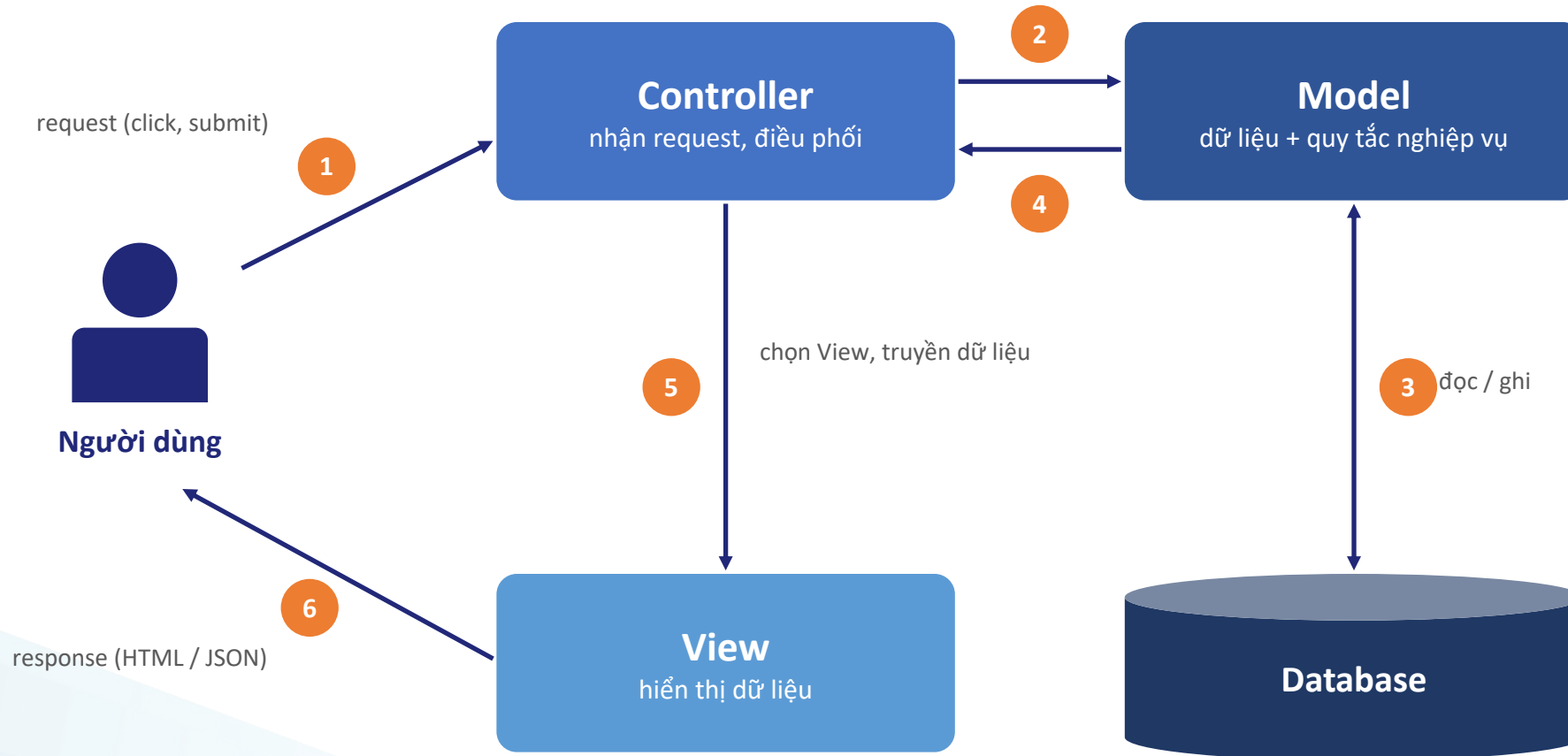
Layer: phân tách logic trong mã nguồn. Tier: phân tách vật lý khi triển khai, mỗi tier có thể chạy trên máy chủ riêng.

Lợi ích

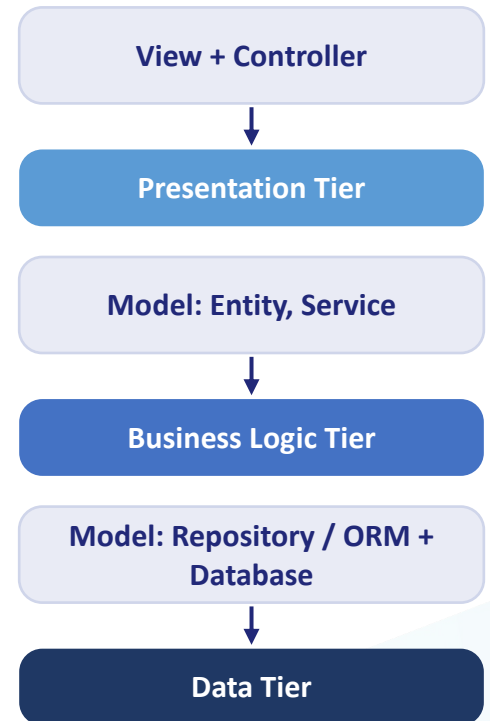
Độc lập giữa các tầng, kiểm thử được từng phần, dễ bảo trì và thay thế công nghệ của một tầng.

2. Các công nghệ lập trình hiện đại

2.2. Mẫu MVC (Model-View-Controller) trong ứng dụng web



Ánh xạ sang 3 tầng



MVC là mẫu tổ chức mã nguồn,
không phải một kiến trúc 3 tầng.

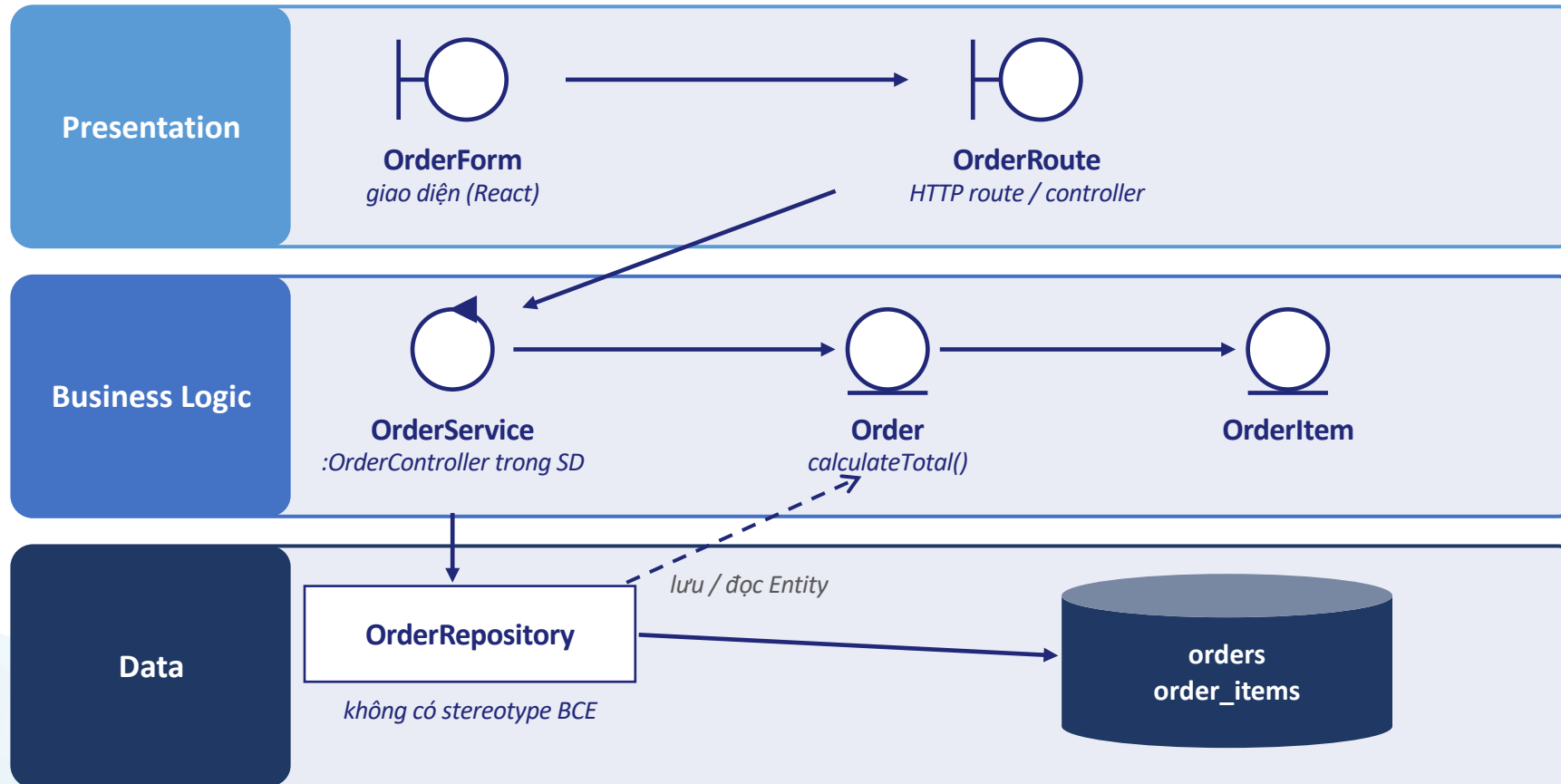
2. Các công nghệ lập trình hiện đại

2.2. Kiến trúc phân tầng (Layered Architecture)

Tầng kiến trúc	Lớp BCE tương ứng (gần đúng)	Trách nhiệm trong hệ thống	Ví dụ
Presentation	<<boundary>> Boundary	Hiển thị dữ liệu cho người dùng; thu nhận và chuyển đổi input; điều hướng màn hình; validate phía client (định dạng, bắt buộc điền); nhận request HTTP	OrderForm.jsx: form tạo đơn; InvoiceView: màn hình xem hoá đơn; route / HTTP controller của framework (Express router, Spring @RestController)
Business Logic	<<control>> Control và <<entity>> Entity (đối tượng miền)	Validate nghiệp vụ phía server; áp dụng quy tắc kinh doanh; điều phối giao dịch; xử lý exception nghiệp vụ; tính toán (giá, chiết khấu, thuế)	<<control>> OrderController trong SD, cài đặt thành OrderService; DiscountService: tính chiết khấu; <<entity>> Order: dữ liệu + calculateTotal()
Data	Không có lớp BCE riêng (lưu trữ trạng thái của các <<entity>>)	Ánh xạ đối tượng sang bảng CSDL; thực hiện CRUD; quản lý kết nối CSDL; truy vấn có điều kiện; đảm bảo tính toàn vẹn dữ liệu	OrderRepository: CRUD đơn hàng; UserDAO: truy cập bảng users; Prisma/Hibernate mapping cho Order

2. Các công nghệ lập trình hiện đại

2.2. Minh họa: các lớp BCE của UC “Tạo đơn hàng” nằm ở tầng nào?



Entity là đối tượng miền: giữ dữ liệu và quy tắc nghiệp vụ (vd `Order.calculateTotal()`). Entity không phải Repository/DAO.

Repository / DAO thuộc tầng Data, chịu trách nhiệm lưu và đọc Entity từ CSDL.

Control trong SD thường được cài đặt thành Service. Controller của framework web thuộc tầng Presentation.

Boundary gồm cả giao tiếp với hệ thống ngoài (cổng thanh toán, email), không chỉ giao diện.

BCE (phân tích) và 3 tầng (kiến trúc) chỉ tương ứng gần đúng, không phải 1-1.

3. Ánh xạ từ thiết kế sang mã nguồn

3.1. Ánh xạ từ Class Diagram sang mã nguồn

Element trong Class Diagram	Tương ứng trong Code	Lưu ý
<<class>> Order	class Order { } hoặc interface Order	Tên class phải khớp chính xác với Class Diagram để đảm bảo traceability
Attribute: totalAmount: Decimal	private BigDecimal totalAmount (Java); total_amount DECIMAL(10,2) trong CSDL	Kiểu dữ liệu phải tương thích giữa ngôn ngữ và CSDL; tiền tệ dùng BigDecimal/Decimal, không dùng Float/Double
Operation: calculateTotal(): Decimal	method calculateTotal(): BigDecimal (Java) / Decimal (Python, Prisma)	Nếu operation thuộc <<control>> class → method trong Controller; nếu thuộc <<entity>> → trong Model
Association: Order →OrderItem (1..*)	Array/List: items: OrderItem[] trong class Order;	Quan hệ 1-N trong UML → One-to-Many trong ORM → Foreign Key trong CSDL (order_id trong bảng order_items)

3. Ánh xạ từ thiết kế sang mã nguồn

3.1. Ánh xạ từ Class Diagram sang mã nguồn

Element trong Class Diagram	Tương ứng trong Code	Lưu ý thực tiễn
Inheritance: VIPOrder extends Order	class VIPOrder extends Order { } trong Java/TypeScript; hoặc Single Table Inheritance / Table Per Class trong ORM	Kế thừa trong UML = kế thừa trong OOP; cần quyết định chiến lược ánh xạ sang CSDL (1 bảng hay 2 bảng)
<<interface>> IPayable với method pay()	interface IPayable { pay(): void } trong TypeScript/Java; hoặc Abstract Base Class trong Python	Interface trong UML → Interface trong ngôn ngữ hỗ trợ (Java, TypeScript, C#); Python dùng ABC
Dependency: Controller → Repository	import / inject dependency trong Controller: constructor(private repo: OrderRepository) { }	Dependency Injection là pattern chuẩn, không tạo instance trực tiếp trong Controller (new OrderRepository())

3. Ánh xạ từ thiết kế sang mã nguồn

3.2. Ánh xạ từ Sequence Diagram sang mã nguồn

Element trong Sequence Diagram	Tương ứng trong Code	Ví dụ cụ thể
Lifeline :OrderController	Instance của class OrderController được tạo ra và inject vào route handler	<code>const controller = new OrderController(orderRepo, productRepo);</code>
Synchronous message: placeOrder(orderData)	Lời gọi hàm đồng bộ (hoặc await nếu async): <code>const result = await controller.placeOrder(orderData);</code>	Tên hàm trong code phải đúng với tên message trong SD
Return message: order	Return value: <code>return order;</code> hoặc kết quả của await expression	Kiểu dữ liệu của return value phải khớp với type đã định nghĩa trong Class Diagram
Alt fragment [stock available] / [else]	Câu lệnh điều kiện: <code>if (stockAvailable) { ... } else { return ErrorResponse; }</code>	Mỗi guard condition trong Alt frame → một nhánh if/else hoặc early return

3. Ánh xạ từ thiết kế sang mã nguồn

3.2. Ánh xạ từ Sequence Diagram sang mã nguồn

Element trong Sequence Diagram	Tương ứng trong Code	Ví dụ cụ thể
Loop fragment [for each item in order]	Vòng lặp: <code>for (const item of order.items) { ... }</code> hoặc <code>.map()</code> , <code>.forEach()</code>	Điều kiện lặp trong SD → điều kiện trong vòng lặp code
Opt fragment [if customer is VIP]	Câu lệnh if đơn: <code>if (customer.isVIP) { applyDiscount(); }</code>	Opt frame chỉ có nhánh true, không có else
Ref frame: <code>validateOrder</code>	Gọi method riêng biệt: <code>this.validateOrder(orderData)</code> , method được định nghĩa trong cùng class	Ref frame gợi ý tách logic thành method con, thực hiện đúng theo SRP
Self-call message: <code>calculate()</code>	Method tự gọi chính nó hoặc gọi method private khác trong cùng class: <code>this.calculate()</code>	Biểu diễn bằng mũi tên cong quay về chính lifeline trong SD

4. Tiêu chuẩn chất lượng mã nguồn

- ❑ Chất lượng mã nguồn không được đo chỉ bằng việc “code chạy được”.
- ❑ Theo Robert C. Martin (Clean Code, 2008), một tài liệu tham khảo được sử dụng rộng rãi trong ngành công nghiệp phần mềm, “code chạy được là điều kiện cần, nhưng không phải điều kiện đủ. Code tốt là code mà một lập trình viên khác có thể đọc và hiểu được trong vòng vài phút mà không cần hỏi tác giả”.
- ❑ Phần này giới thiệu các nguyên tắc thiết yếu và liên hệ với tiêu chuẩn ISO/IEC 25010.

4. Tiêu chuẩn chất lượng mã nguồn

4.1. Bốn nguyên tắc lập trình cốt lõi

- ❑ Nguyên tắc 1: Single Responsibility Principle (SRP)
- ❑ Nguyên tắc 2: DRY (Don't Repeat Yourself)
- ❑ Nguyên tắc 3: Đặt tên có nghĩa (Meaningful Naming)
- ❑ Nguyên tắc 4: Xử lý lỗi rõ ràng (Explicit Error Handling)

4. Tiêu chuẩn chất lượng mã nguồn

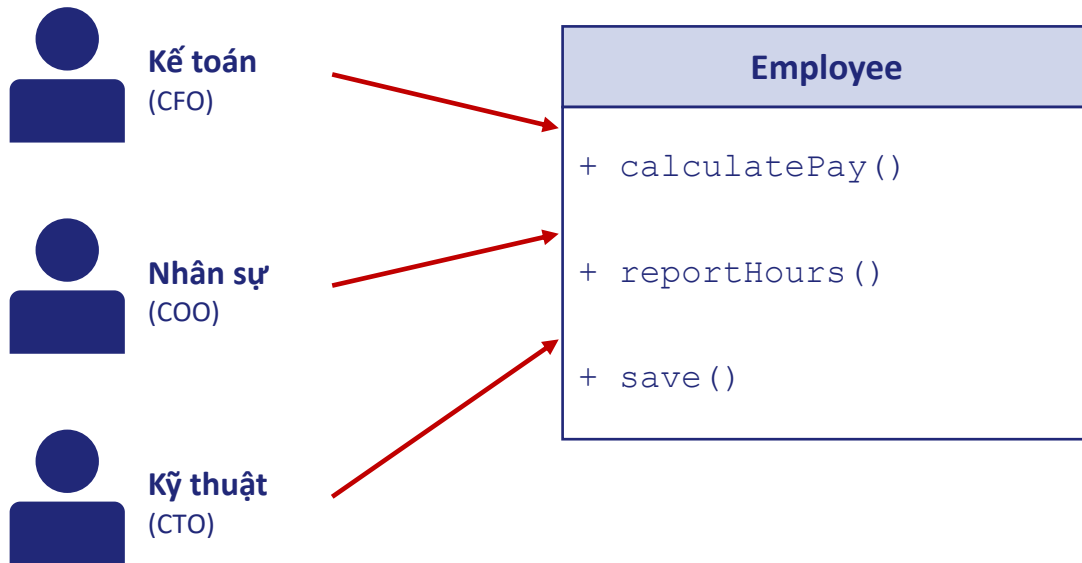
4.1.1. Nguyên tắc 1: Single Responsibility Principle (SRP)

- ❑ Nguồn gốc: Nguyên tắc đầu tiên trong SOLID principles (Robert C. Martin, 2000).
- ❑ Phát biểu chính xác: “Một module nên có một, và chỉ một lý do để thay đổi”.
- ❑ Trong thực tiễn: mỗi class, mỗi method, mỗi module chỉ nên đảm nhiệm một trách nhiệm cụ thể. Nếu cần giải thích trách nhiệm của một class bằng từ “và” (class này làm A VÀ B VÀ C), đó là dấu hiệu vi phạm SRP

4. Tiêu chuẩn chất lượng mã nguồn

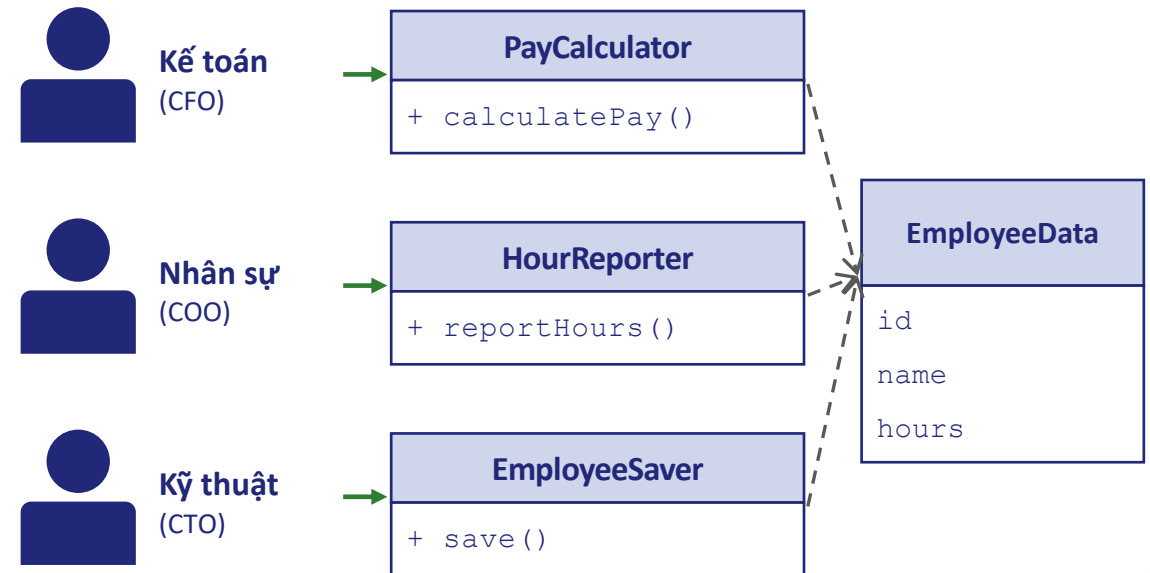
4.1.1. Ví dụ SRP theo actor (Robert C. Martin, Clean Architecture, 2017)

✗ Vi phạm SRP: một class phục vụ ba actor



Kế toán đổi cách tính lương có thể làm hỏng báo cáo giờ công của Nhân sự, vì cả hai cùng sửa một class.

✓ Tách theo actor



Mỗi class chỉ còn một lý do để thay đổi: yêu cầu của đúng một actor. Dữ liệu dùng chung tách thành **EmployeeData**.

4. Tiêu chuẩn chất lượng mã nguồn

4.1.2. Nguyên tắc 2: DRY (Don't Repeat Yourself)

- ❑ Mỗi mẫu kiến thức phải có một biểu diễn duy nhất, không mập mờ và có thẩm quyền trong một hệ thống.
- ❑ Trong thực tiễn lập trình:
 - ✓ Nếu cùng một đoạn logic xuất hiện ở hai nơi khác nhau, khi có thay đổi yêu cầu, lập trình viên phải nhớ cập nhật cả hai nơi, và thường quên một nơi.
 - ✓ Đây là nguồn gốc của rất nhiều bug regression (lỗi xuất hiện lại sau khi đã sửa).

4. Tiêu chuẩn chất lượng mã nguồn

4.1.3. Nguyên tắc 3: Đặt tên có nghĩa (Meaningful Naming)

- ❑ Bất kỳ ai cũng có thể viết code mà máy tính hiểu được.
- ❑ Lập trình viên giỏi là người viết code mà con người có thể hiểu được.
- ❑ Tên biến, tên hàm, tên class là hình thức tài liệu hoá tốt nhất và bền vững nhất, nó luôn tồn tại cùng với code, không thể bị lạc mất như file Word tài liệu bên ngoài.

4. Tiêu chuẩn chất lượng mã nguồn

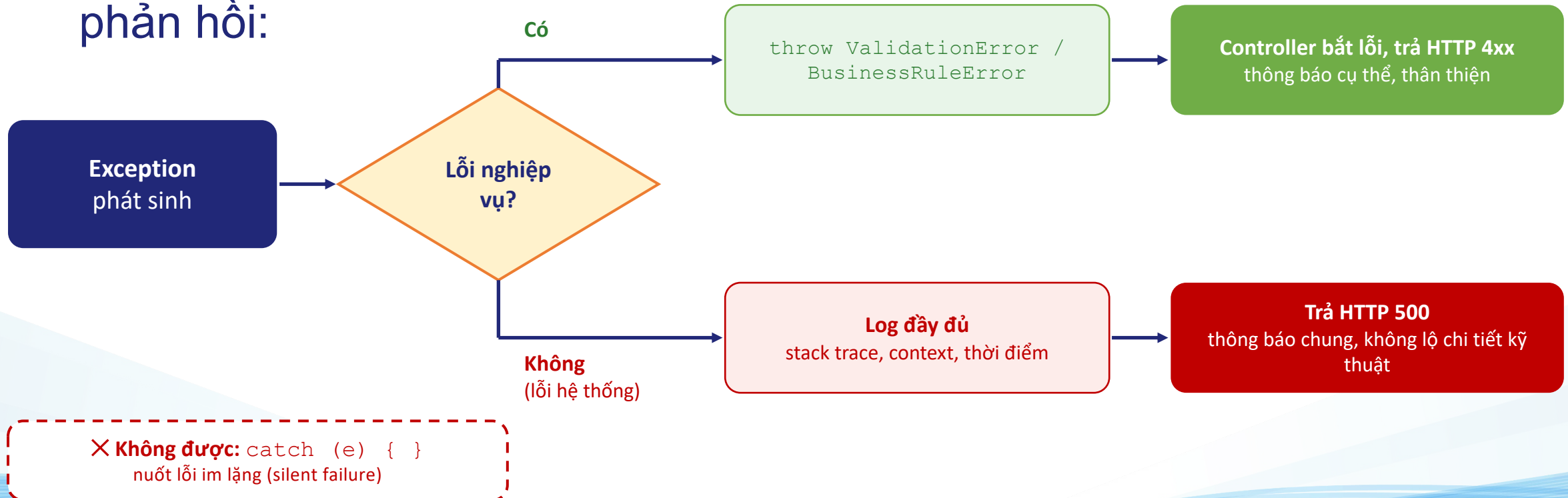
4.1.3. Nguyên tắc 3: Đặt tên có nghĩa (Meaningful Naming)

Loại định danh	Convention	Ví dụ đúng	Ví dụ sai và lý do sai
Biến / Tham số	camelCase; tên danh từ; mô tả nội dung	totalAmount, customerId, orderItems	ta (không rõ), x1 (vô nghĩa), data (quá chung chung)
Hàm / Method	camelCase; bắt đầu bằng động từ hành động; mô tả điều hàm LÀM	calculateTotal(), getOrderById(), validateInput(), sendConfirmationEmail()	calc() (viết tắt khó hiểu), order() (không rõ hành động), process() (quá mơ hồ)
Class	PascalCase; danh từ hoặc danh từ ghép; mô tả điều class LÀ	OrderController, VIPCustomer, PaymentGateway	ordercontroller (không theo convention), Manager (quá chung, không rõ quản lý gì)
Hằng số	SCREAMING_SNAKE_CASE; mô tả rõ ý nghĩa	MAX_RETRY_COUNT, TAX_RATE_PERCENT, VIP_DISCOUNT_THRESHOLD	MRC (viết tắt), 0.1 (magic number không có tên)
Boolean variable	camelCase; bắt đầu bằng is/has/can/should	isVIP, hasDiscount, canCheckout, isOrderConfirmed	flag (không rõ nghĩa), status (mơ hồ), check (mơ hồ)
File / Module	kebab-case (JavaScript), snake_case (Python), PascalCase (Java)	order-controller.js, discount_service.py, OrderRepository.java	OrdCtrl.js (viết tắt), Helper.js (quá chung), util.py (mơ hồ)

4. Tiêu chuẩn chất lượng mã nguồn

4.1.4. Nguyên tắc 4: Xử lý lỗi rõ ràng (Explicit Error Handling)

- ❑ Không bao giờ để exception bị bỏ qua im lặng (silent failure).
- ❑ Phân biệt rõ hai loại lỗi cần xử lý khác nhau hoàn toàn về cách thức phản hồi:



4. Tiêu chuẩn chất lượng mã nguồn

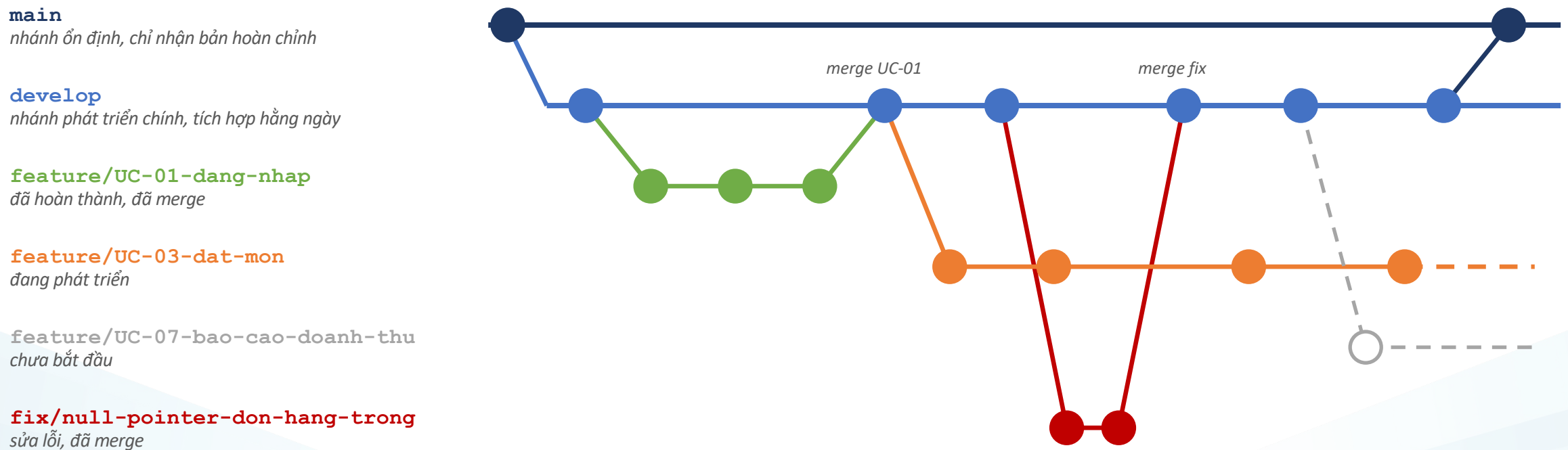
4.1.4. Nguyên tắc 4: Xử lý lỗi rõ ràng (Explicit Error Handling)

Loại lỗi	Định nghĩa	Cách xử lý đúng	Ví dụ
Lỗi nghiệp vụ (Business Error)	Người dùng vi phạm quy tắc nghiệp vụ; hệ thống hoạt động bình thường, chỉ input không hợp lệ	Throw custom exception (NotFoundError, ValidationError, BusinessRuleError); bắt ở Controller; trả về thông báo cụ thể, thân thiện cho người dùng	Đặt sân đã có người đặt; mượn sách khi đang bị khoá tài khoản; thanh toán số tiền âm
Lỗi hệ thống (System Error)	Infrastructure thất bại: CSDL ngắt kết nối, network timeout, file không tìm thấy, memory hết	Log đầy đủ (stack trace, context) để debug; trả về thông báo chung chung cho người dùng (không lộ chi tiết kỹ thuật vì lý do bảo mật)	Database connection timeout; Redis unreachable; Disk full; External API unavailable

4. Tiêu chuẩn chất lượng mã nguồn

4.2. Quản lý phiên bản với Git

Quản lý phiên bản (version control) là kỹ năng bắt buộc của mọi kỹ sư phần mềm chuyên nghiệp.



Mô hình phân nhánh đề xuất cho nhóm 4-5 người: mỗi UC một nhánh feature; develop tích hợp hằng ngày; main chỉ nhận bản ổn định.

4. Tiêu chuẩn chất lượng mã nguồn

4.2. Quản lý phiên bản với Git

Quy ước viết Commit Message theo chuẩn Conventional Commits.

Conventional Commits: quy ước viết commit message

Format: <type>(<scope>): <mô tả ngắn gọn> (#UC-XX)

Types thường dùng:

- feat - thêm tính năng mới
- fix - sửa lỗi
- refactor - tái cấu trúc code (không thêm tính năng, không sửa lỗi)
- test - thêm hoặc sửa test
- docs - cập nhật tài liệu
- chore - cấu hình, build, dependency

Ví dụ commit message tốt:

```
feat(order): implement UC-03 addItem - business logic layer (#UC-03)
feat(order): implement UC-03 addItem - OrderRepository with Prisma (#UC-03)
feat(order): implement UC-03 addItem - OrderForm UI component (#UC-03)
fix(order): null pointer khi giỏ hàng trống trước khi checkout (#UC-03)
refactor(discount): tách DiscountService ra khỏi OrderController (SRP)
test(order): thêm unit test cho DiscountService.calculate() (#UC-08)
```

*# Đặc tả Conventional Commits 1.0 chỉ định nghĩa feat và fix; các type còn lại theo quy ước Angular.
Hậu tố (#UC-XX) là quy ước riêng của môn học, dùng để truy nguyên commit về Use Case.
Thay đổi phá vỡ tương thích: thêm "!" (feat(api)!: ...) hoặc footer "BREAKING CHANGE: ...".*

4. Tiêu chuẩn chất lượng mã nguồn

4.3. ISO/IEC 25010: Mô hình chất lượng phần mềm

ISO/IEC 25010:2023 định nghĩa chín đặc tính chất lượng sản phẩm (bản 2011 có tám). Ở giai đoạn cài đặt, quyết định viết code ảnh hưởng trực tiếp đến ít nhất năm đặc tính được tô đậm dưới đây:

Functional Suitability

Phù hợp chức năng

Performance Efficiency

Hiệu năng

Compatibility

Tương thích

Interaction Capability

Khả năng tương tác (đổi tên từ Usability)

Reliability

Độ tin cậy

Security

Bảo mật

Maintainability

Dễ bảo trì

Flexibility

Linh hoạt (đổi tên từ Portability)

Safety

An toàn (mới trong bản 2023)

4. Tiêu chuẩn chất lượng mã nguồn

4.3. ISO/IEC 25010: Mô hình chất lượng phần mềm

Đặc tính ISO/IEC 25010:2023	Biểu hiện trong mã nguồn	Vi phạm phổ biến cần tránh
Functional Suitability, đặc tính con: Functional Correctness (Đúng đắn chức năng)	Code cho ra kết quả đúng với mọi input hợp lệ và xử lý đúng mọi trường hợp ngoại lệ đã đặc tả trong UC Spec	Logic tính toán sai (làm tròn sai, sai dấu phẩy); thiếu xử lý edge case (giỏ hàng trống, input rỗng, số âm)
Maintainability (Dễ bảo trì)	Code có thể đọc và sửa bởi người khác; tuân thủ SRP, DRY, naming conventions; không có magic number hay magic string	God class (một class 1000+ dòng làm mọi thứ); magic number (if (type == 2) thay vì if (type == USER_TYPE.VIP)); code không có comment cho logic phức tạp
Performance Efficiency (Hiệu năng)	Không có N+1 query problem; không load toàn bộ bảng để filter phía application; dùng pagination; dùng index	for (item of items) { await db.query('SELECT ...') }, N+1 query; SELECT * FROM orders, load tất cả dữ liệu không cần thiết

4. Tiêu chuẩn chất lượng mã nguồn

4.3. ISO/IEC 25010: Mô hình chất lượng phần mềm

Đặc tính ISO/IEC 25010:2023	Biểu hiện trong mã nguồn	Vi phạm phổ biến cần tránh
Security (Bảo mật)	Validate mọi input phía server; dùng parameterized query / ORM để tránh SQL injection; không hardcode credential; hash mật khẩu	Hardcode API_KEY = "abc123" trong source code; dùng string concatenation để tạo SQL query; lưu mật khẩu dạng plain text
Reliability (Độ tin cậy)	Xử lý lỗi rõ ràng theo hai loại; không crash khi input bất thường; có logging để debug khi sự cố xảy ra trên production	Catch-and-ignore exception; không validate null trước khi gọi method trên object; không có error logging